

Trabalho Prático Final - Listado

1. Visão de Projeto

1.1. Escopo do Software

1.1.1. Descrição do Problema

O desafio principal desse projeto é proporcionar ao usuário o gerenciamento financeiro de suas compras, possibilitando a criação prévia de listas, gerenciamento de itens, suas medidas, calcular preço por unidades de medida e fatores de conversão para serem atualizadas convenientemente e prover análises temporais de custo.

O intuito é substituir atividades manuais e dispersas por um sistema centralizado e automatizado para o gerenciamento de compras, aliado à análise de preços.

1.1.2. Objetivo do Projeto

- Implantar um sistema mobile para gerenciar listas de compras e seus respectivos itens, calculando automaticamente o valor total da lista e persistindo constantemente os diversos estados das listas;
- A persistência de dados será feita localmente;
- Item é uma classe cujos objetos são gerenciados individualmente em um catálogo e reutilizados em mais de uma lista;
- Uma lista pode ter um limite orçamentário;
- A conclusão de uma lista é uma compra finalizada e imutável;
- Ocorrências de itens em uma lista são associativas, não refletindo os atributos de um item no catálogo;
- A análise de preço para cada item é realizada obtendo o valor de um item para cada compra concluída nas datas dentro de um intervalo de tempo selecionado.

1.1.3. Fora de Escopo

- Conexão com APIs de terceiros, devido a ausência de serviços disponíveis;
- Web Scraping de portais de preços, devido ser um projeto de Engenharia de Dados de porte considerável para ser entregue em tempo hábil.

1.1.4. Resultados Esperados

- Um sistema robusto e funcional para que um usuário possa gerenciar seus gastos de maneira eficiente e simples;
- Análises de flutuações de preços para os itens cadastrados pelo usuário;
- Interface leve e intuitiva, proporcionando uma experiência leve e fluída.

1.2. Atores do Sistema

- a. **Usuário:** Quem possui a necessidade de controlar seus gastos financeiros e precisa de uma ferramenta para maior comodidade. Utiliza o sistema para gerenciar itens, listas, compras e acompanhar seus aspectos históricos.

1.3. Requisitos Funcionais

Nome	Descrição
RF1	O sistema deverá permitir o cadastro de itens com atributos como nome,

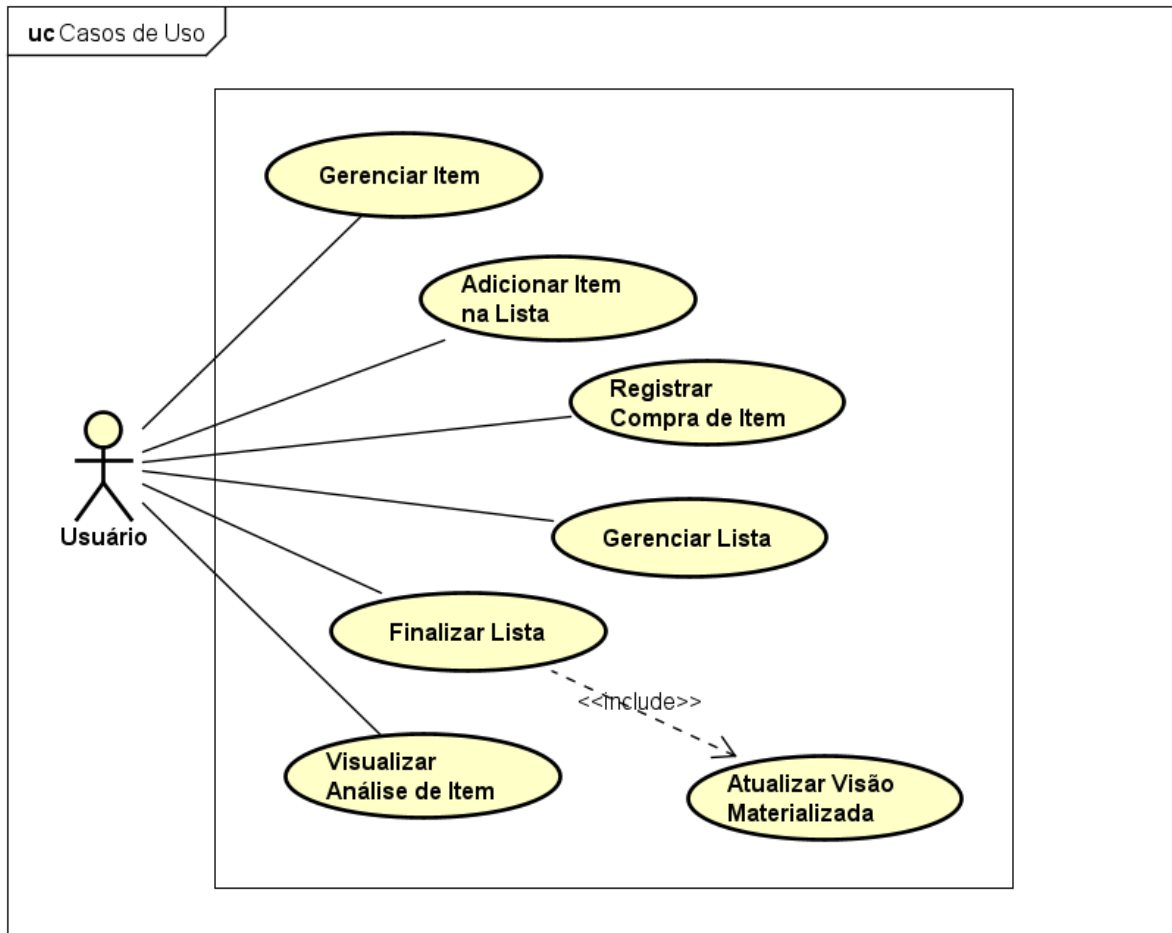
	categoria e unidade de medida.
RF2	O sistema deverá possibilitar a edição e exclusão lógica, garantindo que itens inativos não apareçam em novas listas, mas sejam preservados em históricos de compras passadas.
RF3	O sistema deverá calcular automaticamente o preço por unidade de medida considerando fatores de conversão definidos pelo sistema.
RF4	O sistema deverá permitir a criação de listas de compras contendo itens previamente cadastrados.
RF5	O sistema deverá calcular automaticamente o valor total da lista com base nos itens e suas quantidades.
RF6	O sistema deverá permitir a edição de listas em andamento.
RF7	O sistema deverá bloquear a edição de uma lista após concluída.
RF8	O sistema deverá manter o histórico de listas concluídas.
RF9	O sistema deverá permitir a visualização de histórico de preços de cada item em compras concluídas.
RF10	O sistema deverá gerar análises temporais de custo após a conclusão de cada lista, mostrando variações de preços em intervalos de tempo.
RF11	O sistema deverá permitir a pesquisa dos itens para serem adicionados à lista.

1.4. Requisitos Não Funcionais

Nome	Descrição
RNF1	O sistema deverá responder às ações de interação em menos de 100ms e operações de persistência simples em menos de 500ms.
RNF2	O sistema deverá suportar o cadastro de até 10.000 itens sem perda significativa de desempenho.
RNF3	O sistema deverá realizar cálculos de preço e conversão de unidades em tempo real, com latência abaixo de 100ms.
RNF4	O sistema deverá possuir uma interface intuitiva, adequada para usuários sem conhecimento técnico, com taxa de sucesso acima de 75%.
RNF5	O sistema deverá oferecer feedback visual e/ou sonoro para ações críticas.
RNF6	O sistema deverá garantir a persistência dos dados mesmo após fechamento inesperado do aplicativo.
RNF7	O sistema deverá impedir a edição de listas concluídas, garantindo imutabilidade das compras finalizadas.
RNF8	O sistema deverá ser compatível com dispositivos móveis rodando sistema operacional Android 10 ou superior.

2. Casos de Uso

2.1. Diagrama de Casos de Uso



2.2. Formato Estruturado dos Casos de Uso

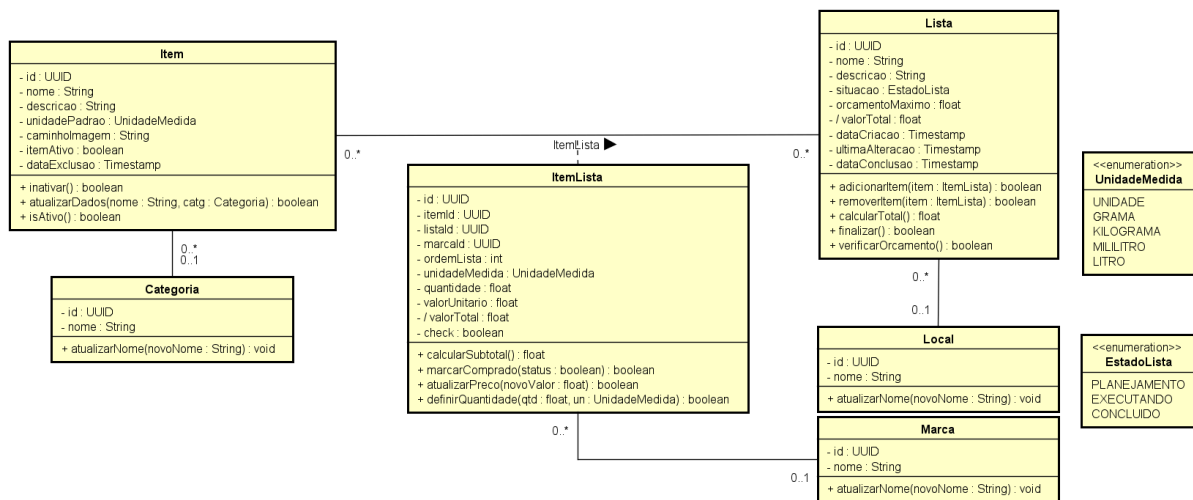
Nome do Caso de Uso	UC05 – Finalizar Lista
Ator Principal	Usuário
Ator Secundário	Sistema
Resumo	Ao finalizar uma lista, o sistema valida a integridade da compra, bloqueia futuras edições para fins de histórico e dispara o processamento assíncrono das estatísticas de consumo.
Pré-Condições	1. Deve existir uma lista com o status "EXECUTANDO".

	2. A lista deve conter pelo menos um item marcado como "check".
Pós-Condições	a. O status da lista passa a ser "CONCLUIDA". b. Os dados da compra tornam-se imutáveis (Read-Only). c. As tabelas estatísticas do módulo de Análise são atualizadas com os novos preços praticados nesta compra.
Ações do Ator	Ações do Sistema
1. Acessa a tela de detalhe de uma lista em andamento.	
2. Seleciona a opção "Finalizar Compra".	
	3. Verifica se todos os itens da lista foram marcados ou se há itens pendentes.
	4. Exibe um resumo da compra e solicita confirmação.
5. Confirma a finalização.	
	6. Altera o estado da lista para "CONCLUIDA".
	7. Bloqueia a edição de todos os itens vinculados a esta lista.
	8. Executa o caso de uso incluso "Atualizar Visão Materializada", processando os dados para o módulo de Análise.
	9. Exibe uma mensagem de sucesso e redireciona o usuário para a tela de "Histórico de Listas".
Fluxo Alternativo I – AF01: Itens Pendentes	
Ações do Ator	Ações do Sistema
	4a. Alerta o usuário: "Existem itens não marcados. Deseja removê-los ou mantê-los como não comprados?"

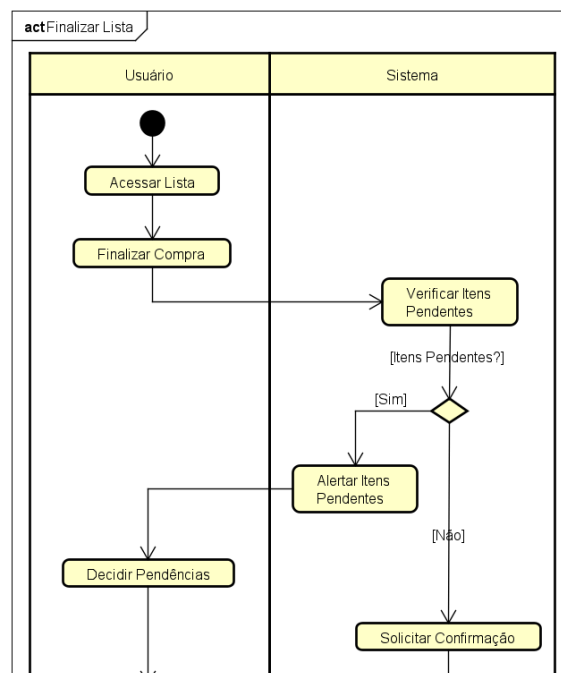
4b. Seleciona a opção desejada.	
Fluxo Alternativo II – AF02: Cancelamento	
Ações do Ator	Ações do Sistema
	5a. Cancela a operação.
	5b. A lista permanece com o status "EXECUTANDO" e é editável.

3. Classes do Sistema

3.1. Diagrama de Classes Físico



4. Diagrama de Atividades



5.2.2. Arquitetura Interna dos Módulos (Micro-Arquitetura)

Internamente, cada módulo implementa uma variação robusta do padrão MVC com Inversão de Dependência, estruturada em três camadas horizontais:

a. Presentation:

- Composta por View e Controller.
- Isola a interface do usuário da lógica de negócios. O Controller orquestra as requisições, delegando o processamento para a camada de serviço.

b. Model:

- Entities: Classes POJO que representam os dados.
- Services: Classes que encapsulam as Regras de Negócio. É a única camada que orquestra as operações lógicas.
- DAO Interfaces: Contratos que definem as operações de persistência, mas não sua implementação.

c. Data:

- Contém as implementações concretas que acessam o banco de dados.
- Inversão de Dependência (DIP): O módulo Data depende do Model implementando suas interfaces, e não o contrário. Isso garante que o domínio de negócio seja alheio à tecnologia de banco de dados.

5.2.3. Padrões de Comunicação e Dependência

a. Dependência Hierárquica

Para os módulos transacionais, aplica-se uma Transitividade Unidirecional:

`Lista` → depende de → `ItemLista` → depende de → `Item`.

- Reflete a relação de composição forte.
- Garante um grafo de dependências acíclico. Módulos de nível superior orquestram módulos inferiores através da injeção de seus Serviços ou DAOs.

b. Padrão Observer e Desacoplamento

O módulo de Análise precisa processar dados de compras, mas o módulo de Lista não deve depender da existência de um módulo de relatórios, pois evita acoplamento bidirecional.

- O módulo Lista define uma interface de contrato.
- O módulo Analise implementa essa interface.
- Quando uma lista é finalizada, o sistema notifica os observadores registrados.

O módulo Lista permanece funcionalmente isolado, desconhecendo quem consome seus eventos.

5.2.4. Estratégia de Performance Analítica

Para otimizar a experiência do usuário nos relatórios, o sistema implementa um modelo simplificado de Command Query Responsibility Segregation (CQRS):

1. Write Model: Ocorre no momento da finalização da lista. Através do evento disparado pelo Observer, o módulo de Análise captura os dados brutos.
2. Processamento Assíncrono/Lógico: O serviço de análise normaliza unidades, calcula variações de preço e consolida as estatísticas.
3. Read Model: Os dados processados são persistidos em uma estrutura otimizada dentro do módulo Analise.

Essa abordagem transfere o custo computacional para o momento da escrita , garantindo que a visualização dos gráficos tenha complexidade de leitura constante "O(1)", sem a necessidade de realizar cálculos pesados em tempo real sobre todo o histórico de compras.

5.2.5. Integridade Referencial e Soft Delete

Para manter a consistência dos relatórios históricos, o sistema veta a exclusão física de registros referenciados. Adota-se o padrão Soft Delete:

- Itens excluídos pelo usuário são marcados como inativos no banco de dados.
- Isso os remove das listas de seleção futuras, mas preserva a integridade das listas passadas e dos gráficos de evolução de preços.

6. Princípios de Projeto

A construção do sistema foi guiada por princípios de Engenharia de Software focados na manutenibilidade, testabilidade e baixo acoplamento.

Open/Closed Principle (OCP): O sistema está aberto para extensão, mas fechado para modificação.

- A implementação do padrão Observer no módulo Lista permite que novos comportamentos sejam adicionados ao evento de "Finalizar Lista" sem alterar uma única linha de código do ListaService.
- Adicionar novos "ouvintes" ao processo de compra não gera risco de regressão no código core.

Information Expert: A responsabilidade foi atribuída à classe que possui a informação necessária para cumpri-la.

- O cálculo do valor total de uma lista foi atribuído ao ListaService em conjunto com a entidade Lista, pois é este agregado que detém a coleção de ItemLista.
- Já o cálculo da média histórica de preços foi delegado ao AnaliseService, que detém o acesso aos dados históricos transversais.

